

Predicting Rating Gains in maimai DX

A Player-Specific Statistical Model in the Rasmai Recommender

Cardin Nguyen

cardin.nguyen.ink

September 30, 2026

Updated October 3, 2026

Abstract

Rasmai takes a maimai DX player's scores and turns them into a list of charts worth replaying, ordered by how much rating each replay would add. This paper goes through every calculation behind that list: the game's rating formula, the two-pool best-50 total, a skill curve fitted to each player, a normal model for how much runs vary, picking targets by expected gain, checking the model against recorded plays, note-by-note loss accounting, technique measures read from chart notation (*simai*), trait detection and route planning, plus two optional beta estimates built from other players' scores in aggregate. Then I run all of it on one real export of 554 scored charts. Those numbers are an example and will look different for other players. I ran the export through the project's own code, with the chart databases and notation fetched the same day. It reproduces all 554 stored chart ratings and a best-50 total of 13,579, one under the 13,580 the site shows. After calibrating on 92 recorded plays, the model puts one replay of GIGANTØMAKHIA at a 36% chance of reaching SS, which would add 4 rating points.

1. Introduction

Rasmai is a Discord bot and web dashboard (rasmai.lol) for [maimai DX](#), SEGA's arcade rhythm game. It's named after Ras, a character from the game. It reads a player's scores from maimai DX NET (the [international site](#)), the game's official score site, and answers one question: which chart should this player replay tonight to raise their rating the most? The source is at github.com/Ne-k/Rasmai.

That question has three parts. How does the game turn a score into rating, and which scores count? What score can this particular player realistically reach on a given chart? And how likely is one run to land it? The game only answers the first. The other two are Rasmai's own model, built from the player's history and checked against their later plays.

I explain each step in the order the code runs it. Section 2 covers the system and Section 3 gives the formulas, constants and thresholds. Section 4 applies them to a real player export recorded on 30 September 2026, and Section 5 covers why I made the choices I did and where the model falls short. Source files are linked where they come up, and Appendix A lists them all.

A note on terms. *Achievement* is the percentage score of a play, from 0 to 101%. Rating only counts it up to 100.5%. A chart's *constant* is its hidden internal difficulty, like 13.0 or 13.6. The game only shows it as a level, "13" or "13+". *Rating* is the whole number a play earns from constant and achievement, and a player's *rating* is the sum of their best 50 chart ratings. *Simai* is the plain-text notation maimai charts are written in.

2. System Overview

Rasmai is a Python program with a separate web front end. The Python side has four layers, and all the calculations in this paper live in the third.

1. **Scraping** ([scraping/](#)). A signed-in session reads the player's profile, score tables, recent plays, per-play judgement pages and play counts from maimai DX NET. The score site leaves a lot out, so other sources fill in the gaps:

- chart constants, versions and regions from [otoge-db](#);
 - revised constants, per-chart regions and song aliases from the `dxdata.json` file of [dxrating](#);
 - search aliases from [maimai-chart-browser](#);
 - English titles, videos, unlock notes and area names from [SilentBlue RemyWiki](#);
 - pattern tags written by editors, and chart notation, from [maiノーツ](#);
 - chart notation in bulk from [Maichart-Converts](#).
2. **Storage** ([storage/](#)). Accounts, scores, play history, judgement pages and rating readings are kept per user. Every play the bot has seen gets stored, so history outlives the game's own 50-play list.
 3. **Engine** ([engine/](#)). Pure functions over the stored data: rating and best-50 pools, the play profile, recommendations, route planning, notation measures, and the insights (calibration, traits, judgements, forecast).
 4. **Interfaces** ([bot/](#), [web/](#)). A Discord bot renders results as embeds and posters. A Next.js dashboard calls an internal JSON API that the bot serves on 127.0.0.1:8765.

I try to go easy on the game's servers. A command first checks the profile and recent plays, which takes a few seconds, and rebuilds from the stored copy unless a new play, play count or rating shows up. A full read only happens when something changed, when the stored copy is a week old, or on `/refresh`.

The engine runs the same sequence on every request ([scraper/analyzer.py](#)):

1. **Enrich**. Match each scraped score to its chart, attach the constant, flag whether the chart belongs to the current game version, and work out its rating (`enrich_songs`).
2. **Pool**. Split the ratings into the new and old pools and total the best 50 (`build_best50`).
3. **Profile**. Fit the player's skill curve, spreads and tier offsets from their scores and plays (`build_play_profile`). The model also calibrates itself against recorded plays and fits the player's traits inside this step.
4. **Recommend**. For each chart, pick a target rank, price it, and rank the charts by expected rating gain (`generate_recommendations`).

Each step feeds the next, so a mistake early on, like a wrong constant, carries into every number after it.

3. Methodology

3.1 Ranks and rating coefficients

Achievement maps to a named rank, and every rank has a coefficient. This is the game's DX table as encoded in [rating.py](#) (`RANK_TABLE`).

Rank	Minimum achievement (%)	Coefficient
SSS+	100.5	22.4
SSS	100.0	21.6
SS+	99.5	21.1
SS	99.0	20.8
S+	98.0	20.3
S	97.0	20.0
AAA	94.0	16.8
AA	90.0	15.2
A	80.0	13.6

Rank	Minimum achievement (%)	Coefficient
BBB	75.0	12.8
BB	70.0	12.4
B	60.0	12.0
C	50.0	10.4
D	0	9.5

A score exactly 0.0001 under some boundaries gets its own coefficient: 100.4999 uses 22.2 instead of 21.6, 99.9999 uses 21.4 instead of 21.1, 99.4999 uses 21.0, 98.9999 uses 20.6, 96.9999 uses 17.6 and 59.9999 uses 11.2 (`BORDERLINE_COEFFICIENTS`).

3.2 Rating of one chart

Take constant c and achievement a , capped at 100.5. The chart's rating is the whole number just below the product:

$$R(c, a) = \left\lfloor c \cdot \frac{\min(a, 100.5)}{100} \cdot k(a) \right\rfloor$$

$k(a)$ is the coefficient from the table above. A score that won't parse, or any constant or achievement that isn't positive, is worth 0 instead of raising an error. The coefficient jumps at each rank boundary, so rating is a staircase in achievement, and crossing a boundary is worth a full step. The reverse function `accuracy_for_rating` finds the lowest achievement that reaches the rating you want. It searches the bands from the bottom and nudges by 0.0001 so floor rounding can't get past it.

3.3 Chart constants and versions

The score site only prints a level like "13+". Rasmai looks the constant up in the chart database by (title, chart type, difficulty), ignoring case and falling back to a title stripped of symbols and spacing (`charts.py`). If two songs share a title, the level on the site breaks the tie. From level 7 up, a level covers two bands: "13" is 13.0 to 13.5 and "13+" is 13.6 to 13.9. When a chart is missing from the database I estimate its constant as the middle of its band, so a missing "13+" reads as 13.75 (`level_range`, `enrich_songs`). A raw database level of "13+" with no constant is parsed as 13.6 (`parse_constant`).

A chart is *new* when its version equals the player's current game version. That version is the highest one with at least 8 of the player's scored charts (`detect_current_version`).

3.4 The best-50 total

The rating on a profile is the sum of two pools (`pools.py`):

- **New pool:** the top 15 chart ratings among current-version charts.
- **Old pool:** the top 35 chart ratings among all other charts.

Utage charts are left out, and each chart counts once, at its highest rating. With N the new pool and O the old pool:

$$\text{Rating} = \sum_{i=1}^{15} N_{(i)} + \sum_{i=1}^{35} O_{(i)}$$

$N(i)$ and $O(i)$ are the chart ratings sorted from highest to lowest. Each pool keeps its *cutoff* (the rating in its last slot) and its *next value* (the rating just outside it). Those two numbers decide what a future play is worth.

Marginal gain. For a chart with rating r' after a new score, a pool changes like this (`gain_if`):

$$\Delta = \begin{cases} \max(r', \text{next}) - r_{\text{now}} & \text{chart already in the pool} \\ r' - \text{cutoff} & \text{chart outside the pool and } r' > \text{cutoff} \\ 0 & \text{otherwise} \end{cases}$$

A chart already in the pool gets its whole improvement. A chart outside it only gets what it adds above the cutoff, because it pushes out whatever sits in the last slot. All of this is measured against today's pools. Once several planned improvements land, the cutoffs rise, so the recommender also simulates the pools after all of them (`summarise`). That gives a smaller, more believable "reachable gain" next to the naive sum.

3.5 Rating colours

The total maps to a colour band, following [SilentBlue RemyWiki's rating page](#): white from 0, blue from 1,000, green 2,000, yellow 4,000, red 7,000, purple 10,000, bronze 12,000, silver 13,000, gold 14,000, platinum 14,500, rainbow 15,000 and kiwami 16,000. Gold and up are cut into 250-point steps, four per band at most (`rating_band` , `rating_step`).

3.6 The skill curve

The official formulas say nothing about what a player will score, so Rasmai estimates it from the player's own scores. The result is a curve $E(c)$: the achievement they reach at constant c when a run goes well. The fit (`profile/fit.py`, `profile/model.py`) has five steps.

1. **Bucket.** Each scored chart goes into a bucket of half a constant, `round(2c)/2`.
2. **Summarise each bucket.** A bucket's representative score is its upper quartile if it holds at least 6 charts, and its 65th percentile if it holds fewer. I want what the player does when they're trying, not their careless runs, and a thin bucket gets the more cautious percentile.
3. **Fit a line.** A [weighted least-squares](#) line $L(c)$ through the representatives, each weighted by $\min(n, 10)$, where n is the bucket's chart count. The slope is clamped to $[-2.5, -0.2]$. With fewer than 8 scores or 3 buckets, the fit falls back to $101 - 0.3c$.
4. **Measure between-chart spread.** Take the buckets with at least 3 charts that are within 4 constants of the hardest bucket. In each one, take the upper half of the scores and subtract the bucket's representative. σ_c is the population standard deviation of those residuals, clamped to $[0.4, 2.0]$ (0.9 when there are 4 or fewer). I leave out easy charts because people play them carelessly and they'd inflate it.
5. **Correct locally.** A straight line misses kinks, so $L(c)$ gets blended with a Gaussian [kernel average](#) of the bucket representatives r_j :

$$\hat{L}(c) = \frac{\sum_j w_j r_j}{\sum_j w_j}, \quad w_j = \min(n_j, 10) e^{-\left(\frac{c_j - c}{0.6}\right)^2}, \quad \beta = \frac{W}{W + 3}, \quad W = \sum_j w_j$$

$$E_{\text{raw}}(c) = \beta \hat{L}(c) + (1 - \beta) L(c)$$

Beta goes toward 1 where the player has lots of charts nearby and toward 0 where they have none, so the curve trusts the data wherever there is some. Then three guards apply:

- **Dense ceiling.** Above the hardest bucket with at least 3 charts, the curve loses a further 0.8 points per constant, so a thin top end never promises high ranks.
- **Own ceiling.** Between populated buckets (4 or more charts), the curve can't go more than 0.3 above their interpolated value.
- **Monotone smoothing.** Starting from the bucket with the most charts, the curve can't rise for harder charts or fall for easier ones. This got rid of kinks that once made the model promise more at constant 15 than at 14.6.

The curve is capped at 100.5. Two summary constants come out of it. The *reach* constant is the hardest c with $E(c) \geq 97$ (an S is expected). The *comfort* constant is the hardest with $E(c) \geq 99$, and never less than one level below reach.

Tier offsets. A Master 13 and an Expert 13 aren't the same chart to the same hands. For each difficulty tier with at least 3 scores, the offset is the median of that tier's residuals from the curve, shrunk by $n / (n + 12)$ so a thin tier stays near zero. Offsets are centred on the player's own chart mix and clamped to ± 6 ([profile/build.py](#)). I use the median because it shrugs off a few abandoned runs. The tier-adjusted curve used everywhere below is

$$m(c, \text{tier}) = E(c) + \text{offset}_{\text{tier}} + \text{shift} + \text{shift}_{\text{tier}}$$

where the two shifts come from calibration (Section 3.11) and stay zero until there are enough recorded plays.

3.7 Two kinds of spread

The model keeps two widths, because they answer different questions.

Between-chart spread (`sigma_at`) says how uncertain the score is on a chart the player has never tried, where I don't know how hard it is *for them*. It's $\max(0.3, \sigma_c)$, multiplied by $1 + 0.5(c - \text{dense ceiling})$ above the dense ceiling, raised to at least 0.8 when fewer than 40 charts are scored, multiplied by the calibration scale s from Section 3.11, and capped at 3.0.

Run-to-run spread (`run_sigma_at`) says how far one run on a chart the player already holds lands from the next. Take the difference between two plays of the same chart and the chart's difficulty cancels, which leaves only run variation. From the stored plays, Rasmai takes successive differences Δ on each chart, drops any run more than 5 points under that chart's best as abandoned, and estimates

$$\hat{\sigma} = \frac{1.4826 \cdot \text{median } |\Delta|}{\sqrt{2}}, \quad \sigma_{\text{run}} = \text{clamp}\left(0.5 + (\hat{\sigma} - 0.5) \frac{m}{m + 15}, 0.3, 1.5\right)$$

where m is the number of pairs. The 1.4826 turns a [median absolute deviation](#) into a standard deviation, and dividing by $\sqrt{2}$ turns a difference of two runs into one run. With fewer than 12 pairs it keeps the default of 0.5 ([insights/calibration.py](#), `run_spread`). When it's used, the run spread is at least 0.25, rises to the between-chart spread above the dense ceiling, and gets multiplied by the calibration scale s . I keep the two apart because using the between-chart number on a chart that's already scored made every rank look reachable.

3.8 Expectation on a specific chart

For a chart the player has scored, `chart_expectation` starts from the tier-adjusted curve m and pulls it toward the player's own record. Let b be the best score and n the play count: the larger of the official count and the number of appearances in the recent-play list, and at least 1 if any score exists. Then, in order:

$$\sigma = \sigma_{\text{run}} \sqrt{1 + \frac{1}{n}}, \quad w = \frac{n}{n + 3}, \quad \mu = m - w \max(0, m - b), \quad \mu \leftarrow \max(\mu, b - 0.5 \sigma), \quad \mu \leftarrow \min(\mu, b + h \sigma)$$

The square-root factor widens the spread for a chart played once, since its standing comes from one sample. The next term says a chart played a lot should be expected near its best even when the curve says otherwise. The floor says a best well above the curve marks a chart the player is good at. The cap, with headroom $h = 0$ (`BEST_HEADROOM`), keeps the centre from sitting above a proven score. The source reports that over 246 recorded runs, moving h from 0.3 to 0 called the exact rank right 64% of the time instead of 61% on runs that beat the player's best, and 42% instead of 39% on runs that didn't.

One adjustment comes last. If the chart shows up two or more times in the recent-play list and the latest score beats the oldest, μ rises by $\min(0.4, 0.25 \times \text{that improvement})$. It comes after the cap, so it can lift μ up to 0.4 above the best.

Dropped bests. A best far under the curve on a chart played at most twice is probably one abandoned run, not the chart's real difficulty. It gets dropped when $b < m - \max(2, 2\sigma_c)$, with σ_c the between-chart spread, and both the play count and the chart's runs in the recent-play list are at most 2. That chart is treated as untried: its *first-pass* estimate is $m - 0.35 - 0.4\sigma_c$, with spread σ_c . When the chart look-up shows odds for a chart the player has never played ([planning/predict.py](#)), it widens that spread by another factor of 1.5, because held-out first passes scattered 1.8 times as widely as repeat plays.

3.9 Odds and target ranks

The chance that one run reaches a target achievement t is the upper tail of a [normal distribution](#), where Φ is its cumulative distribution function:

$$P(a \geq t) = \Phi\left(\frac{\mu - t}{\sigma}\right)$$

For each played chart, [picks/targets.py](#) picks one target from the ladder SSS+ (100.5), SSS (100.0), SS+ (99.5), SS (99.0), S+ (98.0), S (97.0), AAA (94.0) and AA (90.0). A rank only counts as a candidate if it passes every test below.

1. It's above the current score and at or below a *ceiling* of $\min(100.5, \mu + s \cdot \max(\sigma, \sigma_c))$, where s is the challenge level's sigma count. I judge reach with the wider spread because how far a chart can be taken is a question about practice, while the odds of one run use σ alone.
2. It sits within a few points of the current score a : at most $a + \min(6, 1.5 + r + s)$, where $r = \max(0, \mu - a)(1 - n/(n+3))$ is the recovery still expected on an unfamiliar chart. A dropped best is held to $\mu + 0.5\sigma$ instead.
3. For S or higher it needs *proof*: it can't exceed, by more than the level's slack, the best score the player holds in any half-level bucket at $c - 0.7$ or above (`best_near`). Someone who has never reached 97% near a 12.5 isn't told to S a 13.
4. Its odds meet the level's minimum, and its rating is higher than the chart's current rating.

Among the ones left, the level's rule picks one:

Level	Sigmas s	Slack	Minimum odds	Rule	Value of a rank
Easier	0.35	0.5	0.30	likely	odds + gain / 10,000
Balanced	1.0	1.0	0.16	value	gain $\times$ odds
Challenging	1.3	2.0	0.08	gain	gain + odds
Long shots	2.2	3.0	0.03	gain	gain + odds

Balanced is the default. It picks the rank that maximises the odds times the rating the chart itself would gain, which isn't always the next rank up.

3.10 Ranking the picks

A played chart's pick score multiplies four quantities ([picks/generate.py](#)):

$$\text{score} = \Delta R \times P \times \alpha \times \phi$$

- ΔR is the best-50 gain from Section 3.4, computed on the target's rating.
- P is the odds from Section 3.9.
- α (**affinity**) rewards charts like the ones the player picks. Recent plays give counts by chart type, genre and difficulty. Each count becomes a multiplier $1 + \gamma \tanh(\text{count} / \text{average} - 1)$, with $\gamma = 0.35$ for type, 0.25 for genre and 0.20 for difficulty, so no factor strays more than γ from 1. The three multiply together.
- ϕ (**freshness**) discounts charts that are already being ground, since the easy points are gone. It's 0.45 for a chart with 3 or more runs in the recent-play list; 0.75 (one run) or 0.6 (two) if its latest run is among the last 5 plays; 0.95 if it's among the last 15; otherwise 1.

The score orders the near misses below. The rating moves themselves are sorted by ΔR first and odds second, since ΔR is a whole number that lots of charts share. Only the Easier level sorts by $\Delta R \times \text{odds}$ instead. An optional beta reading of the player's taste can multiply the odds in that tie-break, and without it the weight is exactly 1. At most 6 unplayed charts show up among the picks.

Near misses. A chart whose target wouldn't move the pools can still be one good run away from entering them. Rasmai asks what achievement gives the pool's cutoff + 1 (`accuracy_for_rating`). If that's within $2 \cdot \max(0.5, \sigma_c)$ of the expected score, the chart is listed as a near miss, ranked below every real gain by $\alpha \rho / (1 + \text{gap})$.

Unplayed charts (`picks/candidates.py`). The search covers Expert, Master and Re:Master charts that are playable in the player's region and not locked, with constants from reach - 1.5 up to the *search ceiling* + 0.3. The search ceiling is the larger of the hardest constant with an S and the reach constant, so one failed attempt at a 15 doesn't open up 15s. The expected first pass is $m - 0.35 - 0.4\sigma_c$. A chart is offered if that estimate is at least 94 (80 while its pool has open slots) and it would enter the pool. Its score is $\Delta R \cdot P \cdot \alpha \cdot \text{proximity} \cdot 0.7$, where $\text{proximity} = 1 / (1 + |c - \text{comfort}| / 3)$, and a new-pool chart scores 1.4 times more while the new pool has open slots. When fewer than 12 rating moves are left, the list carries on with charts worth a first run.

3.11 Calibration against recorded plays

Every stored play that was made with an earlier best on record gets predicted from that earlier best and then compared with what happened (`insights/calibration.py`). For each one, $z = (a - \mu) / \sigma$, with σ at least 0.3, and plays with $z < -4$ are dropped as abandoned. Once there are at least 15 usable plays (n), the model corrects itself:

$$s = 1 + (\text{clamp}(\rho, 0.6, 1.8) - 1) \frac{n}{n + 30}, \quad \rho = 1.4826 \text{ median } |z - \text{median}(z)|$$

$$\text{shift} = \text{clamp}\left(Q_{0.65}(a - \mu) \frac{n}{n + 30}, -2, 2\right)$$

The scale s multiplies both spreads. The shift moves the curve, because the model's centre is meant to be "a good run", so the upper third of the residuals should sit on zero. Each tier with at least 8 plays (m of them) also gets its own shift of $(Q_{0.65} \text{ of the tier} - Q_{0.65} \text{ of all}) \cdot m / (m + 20)$, clamped to ± 2 . Every correction shrinks toward no change while there's little evidence. The result also reports *coverage*, the share of plays with $|z| \leq 1$ (about 68% for a well-sized normal model), and the model's predicted new-best rate against the actual rate.

3.12 Where the points went: note-level losses

A play's judgement page lists, for each note type (tap, hold, slide, touch, break), how many were critical perfect, perfect, great, good or miss. `losses.py` splits the achievement lost across those types. Each note type is worth a share of the 100% base in proportion to its weight:

Note type	Weight
Tap	1
Touch	1
Hold	2
Slide	3
Break	5

With counts n_k and weights w_k , one weight unit is worth $b = 100 / \sum w_k n_k$. A great keeps 80% of a note, a good 50%, and a miss nothing, so the loss on a non-break type is

$$\text{loss}_k = w_k b \left(\frac{\text{great}}{5} + \frac{\text{good}}{2} + \text{miss} \right)$$

Breaks work differently. From the base, a break good costs 3 units and a break miss costs 5. Each break also carries an equal share of a 1% bonus: a critical perfect keeps its whole slice, a great 40%, a good 30% and a miss none. Two judgements come in grades the page doesn't show. A break great keeps 80%, 60% or 50% of the break, so it costs 1, 2 or 2.5 units, and a plain perfect keeps three quarters or half of its bonus slice. The achievement settles which. The code tries every split of the greats, works out from what's left how many perfects kept three quarters, and charges the mix that lands closest. The game cuts the achievement off at four decimal places instead of rounding it, so the true loss can sit up to 0.0001 below the one shown, and the search aims at the middle of that window. Whatever few ten-thousandths the best mix leaves over go with the bonus, so the losses add up to 101 minus the achievement. Over 457 recorded plays in the example and debug exports, a mix fits to within that window on 454. The other three have counts that can't give their achievement at all. On 255 of the 263 plays with a break great, only one mix fits, so the greats' cost comes from the data and isn't assumed. The bonus can be reported by itself, so a player who isn't hunting criticals doesn't look weak at breaks. What a break great costs of the break itself stays with breaks. The arithmetic, including the search over grades, follows [mimai-score-details](#) by SpiritsUnite (Apache-2.0).

Across many plays, [judgements.py](#) compares each type's share of the loss with its share of the points at stake. The ratio is the *tilt*: 1.0 means a type costs what it was worth. With at least 3 plays stored, a type is called weak when its tilt is at least 1.5 and it carries at least 15% of all loss. The same pass reports the share of timing errors that were late, and the loss per play. A stored play whose counts couldn't have given its achievement, like a 26% run with a handful of misses, is left out of this pass (`counts_fit` in `losses.py`), because one record like that outweighs a lot of sound plays.

3.13 Techniques read from chart notation (*simai*)

Editors at `maiノーヅ` have tagged about one chart in ten, almost all Master and above. To reach the rest, Rasmai reads each chart's *simai* notation and measures what the notes ask of the hands ([engine/simai/](#)).

Parsing ([parse.py](#)). A *simai* chart is a run of beat slots separated by commas. `(120)` sets the tempo and `{8}` sets how much of a bar each comma advances. Buttons 1 to 8 run clockwise round the ring, and touch areas are A to E. A slash joins notes struck at the same moment. The parser turns this into a list of notes, each with a time in seconds, a kind (tap, hold, slide, touch, touch hold), a position, and, for holds and slides, a duration, shape, end button and waiting time. If a reading leaves any character unexplained, it's refused instead of guessed at. When a file holds several charts, the one whose note count is nearest the published count is taken.

Measures ([features.py](#), [techniques.py](#)). Every measure is a rate or a share of the chart's notes, so long and short charts compare fairly. The general ones are notes per second, the most notes in any one second (*bursts*), the share of slides that travel in 0.35 s or less, the share of touch notes landing together, the mean ring distance between consecutive notes (*reaching across the screen*), the share of notes struck while an earlier hold or slide is still running (*both hands at once*), and the count of tempo changes. The technique detectors work on struck notes in time order, since a slide's body is followed and not struck:

- A *run* is a stretch of notes at most 0.18 s apart whose gaps stay within 35% of each other.
- A run of 3 or more on one button is a *jack*. A run of 4 or more that alternates between buttons is a *trill*: *on the spot* between two buttons less than 3 apart, *across the screen* between two buttons 3 or more apart. A run of 5 or more with every other note on one button is a *trill against a held button*. Any other run of 8 or more is a *stream*.
- A *bouncing rhythm* is a long-short-long pattern of gaps, each gap at most 0.5 s, whose long-to-short ratio is 1.6 to 3.4.
- *Spinning round the ring* is 5 or more notes, each at most 0.5 s apart, stepping at most 2 buttons the same way round.

- Slide measures count *chained slides* (a star starting where the last ended, within 1.5 s), *delayed slides* (notes struck while a star waits), *fan slides*, *a trill over a slide*, *overlapping loop slides*, *slides at different speeds* (overlapping slides whose durations differ by 2 times or more), *slides fired from one spot* (3 or more from one button within a phrase), and *a slide traced straight back*. Touch measures count *touch sweeps* (3 or more pads in a run) and *touch clusters* (3 or more pads at once).

Naming. A chart earns a measure's label when it sits in the top quarter of the game for that measure, taken over charts of level 12 and above, once at least 40 charts have been read. Three rare measures use a fixed bar instead, because three quarters of charts score zero on them: 1 tempo change, trills across the screen at 0.004, and slides at different speeds at 0.001. The labels join the chart's other attributes as tags.

Validation. A measure only gets a name if it passes two gates recorded in the source: it has to rise with chart level, and it has to run higher on charts the editors tagged for the same thing. The source reports, for example, streams with a level correlation of +0.61 and 4.1 times more common on charts tagged 乱打, and trills at +0.55 and 5.1 times on トリル. Jacks is the weakest named measure, at +0.03 and 1.9 times. Measures that fell with level, like the old slide-shape spin count and the share of simultaneous notes, are still stored but no longer named. A scraped title is matched to its notation exactly first, then by the loose title match of Section 3.3. The source reports that the loose match reaches 95% of the charts a player has played, against 62% for the exact one ([scrapping/simai.py](#)).

3.14 Traits

A *trait* is a chart attribute on which the player scores above or below their own curve ([insights/traits.py](#)). The attributes come from [insights/tags.py](#): chart type, the simai labels of Section 3.13, tempo (130 BPM or under, 210 or over), note count (under 620, 890 or more), era, genre, designer, and editors' pattern tags where they exist. Editor tags that describe fame and not technique, like "notorious", are left out.

1. **Observations.** Each chart's best is one observation, with residual $r = a - m$. Every recorded play is another, and the plays of one chart share one unit of weight, so a ground chart doesn't outvote the rest. Residuals below -12 are dropped, and so are stale bests: an official play count of at most 2 and a gap below the curve of more than $\max(6, 3\sigma_c)$.
2. **Fit.** All tags are fitted together by [ridge regression](#), with penalty 10 per tag, which shrinks a tag on n charts by about $n / (n + 10)$. The fit holds fixed an intercept, whether the row is a play, $\log(1 + \text{plays})$, the constant, and whether an editor tagged the chart at all. A tag needs at least 8 charts to be fitted.
3. **Chance test.** A [permutation test](#) shuffles the tags across charts 400 times. A trait's p-value is $(k + 1) / 401$, where k is the number of shuffles whose offset is at least as large.
4. **Stability test.** The player's charts are split in half at random three times, and each half is refitted. The sign has to hold in both halves every time, with a coefficient of at least 0.1.

A trait is *confirmed* when $p \leq 0.02$, both halves agree, it covers at least 12 charts and its offset is at least 0.4. Chart type, era, genre and designer are measured but never reported as skills. A skill group that isn't confirmed but has an offset of at least 0.25, $p \leq 0.15$ and at least 12 charts is shown as a *leaning*, and leanings are never used by the model.

The note-type losses from Section 3.12 make a second family of traits with the same shape. For each type, the offset is $-(\text{the type's loss rate} - \text{the player's overall rate}) \times \text{the type's points at stake per play}$. A type needs 200 notes. It's confirmed at 25 plays with an offset of at least 0.4, and shown as a leaning from 10 plays at 0.25.

3.15 Forecast and route planning

The rating forecast ([insights/forecast.py](#)) takes the stored rating readings from the last 120 days. It needs at least 3 readings spanning at least 10 days. Its slope is the median over every pair of readings at least a day apart, a [Theil-Sen estimator](#), which steps over the long flat stretches between sessions. The days to the next thousand are $(\text{goal} - \text{current}) / \text{slope}$, dropped if more than 730. A slope under 0.15 per day is called flat.

The route planner ([planning/build.py](#)) builds a path to a goal, by default the next thousand, greedily. It lists every (chart, target rank) pair whose target is above the current score, at most $\mu + p\sigma$ and at most 8 points above the current score, passes the S-or-better proof from Section 3.9, and has odds of at least a floor. Both p and the floor depend on the level: p is 0.4, 1.0, 1.8 or 2.6 and the floor 0.35, 0.2, 0.1 or 0.04, from Easier to Long shots. Unplayed charts from reach - 1 to the search ceiling + 0.3 enter at S or better, priced with a spread of at least $\max(0.5, \sigma_c)$. At each step the planner takes the option with the largest gain $\times$ odds $\times$ affinity, counting gain against pools that already include the earlier steps, and it discounts unplayed charts by 0.7. It stops at the goal or after 40 steps. If it's still short, it may add up to 8 more never-played charts from a wider window, with a lower odds floor, and marks them.

3.16 Two cohort estimates (beta)

Everything above reads one player's own scores. These two beta features are the first thing in Rasmai that looks at other players, so I've kept them apart from the model. Both are off until a player switches them on. Both only display: they never touch ΔR , the odds, the picks or the route. Both live in [engine/cohort.py](#). They work from one best score per player per chart, held in memory and rebuilt at most once an hour ([bot/state/cohort.py](#)). A player with fewer than 30 scored charts isn't counted, and nothing shows at all until 20 players are. A player can opt out under Sharing. Someone who opts out still gets their own result, but nobody else's result uses their scores. What comes out is counts and typical scores, never who they came from.

Observed difficulty. I say a player has beaten a chart when their best is at least 99 (an SS). First I fit one logistic curve over every (player, chart) pair, using the player's rating R_u and the chart's listed constant c_k , both centred, with Newton's method:

$$\text{logit } P(y_{uk} = 1) = a + b \frac{R_u - \bar{R}}{1000} + g(c_k - \bar{c})$$

The slope g comes out negative, since harder charts get beaten less often. Then each chart gets an offset d_k on top of that curve, with a prior that pulls it toward zero:

$$\text{logit } P(y_{uk} = 1) = a + b \frac{R_u - \bar{R}}{1000} + g(c_k - \bar{c}) + d_k, \quad d_k \sim \mathcal{N}(0, 0.7^2)$$

The prior is what stops a chart with few players from claiming much: its offset stays near 0. A positive offset means the chart was beaten more often than its constant says, so I turn the offset back into constant units, with a standard error from the same fit:

$$\hat{c}_k = c_k - \frac{d_k}{|g|}, \quad \text{se}_k = \frac{1}{|g| \sqrt{\sum_u p_{uk}(1 - p_{uk}) + 1/0.7^2}}$$

A chart is shown only if at least 12 players have a score on it. It makes the harder or easier list only if $\hat{c}_k - c_k$ is at least 0.2 and at least one standard error. If the fitted g isn't below -0.05, the fit has nothing to translate with and shows nothing. What this measures is how hard a chart is for the people who played it. Rating is already in the curve, so it isn't fooled by strong players liking a chart. What it can't see is that people play charts that suit them, so the players on any chart do a bit better there than a random player of the same rating would, and a chart that mostly suits the people who choose it can look easier than it is. The standard error only covers sampling noise in this cohort, not the model's own limits, so with a big cohort it gets small while the real error doesn't. That's why the lists also need the 0.2 floor. I treat the whole thing as a second opinion on the listed constant, not a replacement for it.

Players like you. For one player I look for neighbours: other players within 1,500 rating points who share at least 30 scored charts with them. The distance between two players is the median absolute gap between their bests on the shared charts, and I drop anyone farther than 3.0 points. I use the median so a few charts one of them never practised don't push them away. I keep the closest 12, and the feature needs at least 5. A chart is a candidate if at least 4 neighbours have played it, the median of their bests $\tilde{m}_k$ is at least 97, and its

constant sits between reach $- 1.5$ and reach $+ 0.3$. Reach is the constant from Section 3.6, or the hardest chart the player has scored 97 on if there's no curve yet. Call b_k the player's own best on the chart or, if they haven't played it, the score their curve expects there (Section 3.8), or a straight line through their own scores when there's no curve yet. With n_k neighbours on the chart, a candidate's gap is shrunk as if 4 more neighbours had scored exactly b_k :

$$(\tilde{m}_k - b_k) \cdot \frac{n_k}{n_k + 4} \geq 1$$

and the candidates are ranked by that same number. The shrinking is there because the charts with the biggest raw gaps are partly the luckiest medians, and a median of 4 neighbours is luckier than a median of 12. The top 12 are shown, each with how many neighbours it rests on. That median covers only the neighbours who played the chart, and people play what they expect to do well on, so it runs high.

Two more numbers beside them. Each chart on the list, and in the harder and easier lists, also shows the median best of everyone in the cohort who has played it, once at least 5 have. It's a median and not a mean because it sits right next to the neighbours' median, and a few failed attempts drag a mean down. On the debug exports a chart's mean sits 0.85 points under its median on a typical chart and up to 13 under on a few, so a mean would make players like you look further ahead of everyone than they are. It counts everyone who played the chart, the player included, not everyone in the cohort.

Above the list, the player's judgements sit beside their neighbours'. Each neighbour's stored judgement pages become one profile the same way the player's own do (Section 3.12), so a neighbour with 200 plays counts the same as one with 6. For each note type the page shows the player's loss per 100 notes and clean rate (critical or perfect) next to the median of the neighbours who have that type. The loss per play and the share of late hits get the same treatment. A note type, or the late share, only shows when at least 5 neighbours have it, and the whole comparison needs at least 5 neighbours with a profile. Loss per 100 notes isn't a pure measure of accuracy. A note on a short chart is worth more (b in Section 3.12 is 100 over the chart's total weight), so a great on a 400-note chart costs about two and a half times what it does on a 1,000-note chart. Neighbours play charts of a similar level, so it's roughly fair between them, but a player who sticks to longer charts will look a bit cleaner than they are. The clean rate doesn't have that problem.

What I've checked. I haven't run either on a real cohort. The debug exports hold only 5 different players, so both correctly show nothing, since the minimum is 20. What I have is a simulated world with a planted answer: 220 charts, each with a true offset from its listed constant, and players whose scores come from their rating. In it the observed shifts line up with the planted offsets at a rank correlation of 0.74 to 0.81 with 90 players (three seeds), 0.90 to 0.93 with 600 and 0.95 to 0.96 with 2,000. The shifts come out about evenly harder and easier: with 90 players, 106 of the 205 charts shown were harder than listed. A chart with 3 or 11 players isn't shown, one with exactly 12 is, and a thin chart planted two levels too hard is pulled back toward its listed constant. The size of a shift is right too, not just its order. On charts with 40 or more players, a planted offset of 1 comes back as about 0.8 with 90 players, which is the prior doing its job, 0.9 with 600 and 1.0 with 2,000. The average error against the planted offsets is between -0.004 and $+0.03$ levels. Of the charts that made a harder or easier list, 88 to 97% had the right sign and a planted offset of at least 0.1. The standard error is a different story. One standard error covers the planted offset 74 to 77% of the time with 90 players, but only about half the time with 2,000, because by then sampling noise is small next to what the model can't capture: in this world players differ in how steeply difficulty bites them and in their strengths. I also tried refitting the curve with the offsets held in. It moved the RMSE by at most 0.004, so I left it out.

I rechecked the arithmetic against code written separately from the engine, on a fresh simulated world of 150 charts and 120 players. A plain logistic regression from a different starting point gives the same slope g to six decimal places. Solving each chart's offset by bisection instead of Newton's method gives the same $\hat{c}_k$ to within 10^{-14} . A brute-force search finds the same 12 neighbours in the same order, and the per-chart medians, the typical scores and the neighbour counts on every pick match a plain median. I also redrew

who beat each chart 200 times from the fitted probabilities and refitted. The spread of $\hat{c}_k$ across those refits came out at 0.91 of the reported standard error on a typical chart, and between 0.82 and 0.96 for four charts in five. So the standard error is slightly wider than the sampling noise it's meant to cover, which is what the prior should do, and it still leaves out the model's own limits as above.

For Players like you I hid a quarter of each player's scores and asked it about those charts, comparing against the player's own straight-line estimate, the fallback above. On every chart the neighbours had a median for, the median was no closer to what the player scored than that line was (mean absolute error 2.46 against 2.46 points with 150 players, 2.38 against 2.48 with 600), and it ran high by 0.3 to 0.5. On the charts it actually offered, the player then beat their own expected score by a median of 1.3 to 2.6 points and did so 72 to 79% of the time. Across every chart that at least 4 neighbours had played and where their median was at least 97, it was 0.5 to 0.7 points and 62 to 67%, so the gap rule does find charts that suit the player. The shown median still overshoot what the player scored on those charts by 0.5 to 1.7 points, so it isn't a target. The shrinking by 4 helped a little, within the noise, and I kept it because it can't hurt much. All of this is a world built to suit the method, the line is cruder than the curve the dashboard really uses, and real scores will be messier, especially from people who tried a chart once. Until a real cohort says otherwise, both are hints.

4. Worked Example

Everything in this section comes from one Rasmai user, and it only illustrates the method. Another player's curve, spreads, calibration, picks and traits will come out differently. I run the methods of Section 3 on that user's export, `rasmai-scores-20260930-0606.json`, recorded on 30 September 2026 at 06:06. Every figure below comes from calling the project's own functions on the export. Titles are written the way the export stores them, which drops the spaces from a lot of them.

4.1 The example user's data

Item	Value
Scored charts	554 (350 Expert, 158 Master, 32 Advanced, 14 Re:Master)
Charts flagged as current version	122 (version 26 in otoge-db's numbering)
Charts with constant 13.0 or higher	145
Total plays	593
Stored plays	168, of which 107 have a judgement page and 108 have an earlier best on record
Recent-play list	50 plays
Charts with official play counts	91
Rating readings	26, from 3 to 30 September
Rating on the site	13,580 (silver band)

The ranks are spread across the whole ladder: 41 charts at SSS+, 41 at SSS, 43 at SS+, 41 at SS, 75 at S+ and 58 at S, with the other 255 below S.

Scope of the reproduction. The calls below use the export's stored constants and new-chart flags, which I checked against otoge-db and dxrating as fetched on 30 September 2026. Enrichment changed none of the 554 constants or flags. The rerun is the full profile build, with calibration, trait fit and judgement profile included. Chart notation came from Maichart-Converts: 6,995 charts, 437 of which match this player's 554. Three things are left out. `maiノーツ` couldn't be reached, so there are no editor tags and the notation wasn't checked against its published note counts. The optional beta taste model and the two cohort estimates weren't run. The dashboard's numbers can differ where those matter.

4.2 Rebuilding the rating

The export stores a rating for each chart. I recomputed each one with the formula from Section 3.2 using its constant and achievement, and all 554 match. Two rows show the floor at work:

- BreakTheSpeakers, constant 12.9, achievement 100.1567 (SSS, $k = 21.6$): $12.9 \times 1.001567 \times 21.6 = 279.08$, so 279.
- GetUback, constant 12.9, achievement 100.0008 (SSS): $12.9 \times 1.000008 \times 21.6 = 278.64$, so 278.

Here are the example user's 15 new-pool charts. They total 4,054, the cutoff (15th place) is 260, and the next chart outside the pool is worth 257.

Chart	Tier	Constant	Achievement (%)	Rank	Rating
おペンきょうたいむ	Expert	12.4	100.5784	SSS+	279
Let'sミクササイズ!!	Expert	12.4	100.6818	SSS+	279
AiAe	Expert	12.4	100.5860	SSS+	279
BreakTheSpeakers	Expert	12.9	100.1567	SSS	279
GetUback	Expert	12.9	100.0008	SSS	278
Myosotis	Expert	12.0	100.6776	SSS+	270
テトリス	Master	12.0	100.7807	SSS+	270
零號車輛	Expert	12.8	99.7583	SS+	269
7Wonders	Expert	12.8	99.6400	SS+	269
Worlders	Master	12.8	99.6147	SS+	269
モニタリング	Master	11.8	100.6320	SSS+	265
雲外蒼電-DreamingVoltage-	Expert	12.2	100.3777	SSS	264
トレジャーガーデン	Expert	11.7	100.7938	SSS+	263
愛スクリーム!	Re:Master	13.4	97.6384	S	261
CustomizedJustice	Expert	13.0	98.8501	S+	260

The old pool holds 35 charts totalling 9,525, from FREEDOMDiVE(tpzOvercuteRemix) at 288 down to a cutoff of 263 (LostDesire, constant 11.7 at SSS+). The next chart outside the pool is also worth 263. The pools add up to $4,054 + 9,525 = 13,579$, one point under the 13,580 the site shows. Two of the seven other accounts in the debug exports come out exactly one point under as well. Section 5.2 covers what I ruled out.

Two features matter later. The old pool spans constants 11.7 to 13.8, and 16 of its 35 charts are at SSS+, the rank with the biggest coefficient (22.4). And the old pool's cutoff equals its next value, so a chart that enters at exactly 263 gains nothing.

4.3 The fitted curve

The example user's scored charts fall into half-constant buckets. This table lists the buckets at and above 12.0 with the curve the model builds from them.

Bucket	Charts	Typical score (%)	Best (%)	Curve E(c) (%)
12.0	62	100.04	100.89	99.30
12.5	59	99.63	100.72	98.59
13.0	104	97.88	100.66	96.42
13.5	65	91.91	99.23	92.21
14.0	26	89.54	97.51	89.84
14.5	4	83.53	88.05	83.83

Buckets from 10.0 to 11.5 hold 28 to 56 charts each and sit at 99.8 to 100.3 typical, and the 3 charts in the 15.0 bucket sit at 72.3. The straight-line step gives $L(c) = 111.61 - 1.326c$ and a between-chart spread σ_c of 1.87, close to its 2.0 upper clamp. At $c = 13.0$ the line alone gives 94.4, well under the 97.9 this player usually scores there. The buckets around 13.0 carry a combined kernel weight of about 21, so $\beta \approx 0.88$: the local average of about 96.7 dominates, and the blend is 96.4. That's the curve value in the table. It lands below the 13.0 bucket's own 97.9 because the kernel also reaches the 13.5 bucket at 91.9.

From the curve the profile reads three landmarks. The comfort constant is 12.2 (SS expected), the reach constant is 12.8 (S expected), and the hardest chart with an S or better is 13.8. The dense ceiling is 15.0, because even the 15.0 bucket holds 3 charts, so the dense-ceiling drop-off never applies to this player. Tier offsets come out at +1.02 for Expert, +0.16 for Advanced, -2.15 for Master and -1.90 for Re:Master, so an Expert 13 is expected about 3.2 points higher than a Master 13.

The run spread comes from 81 pairs of successive plays among the 168 stored plays. The measured value, about 0.458, is blended with the 0.5 default at weight $81 / 96$, giving 0.464. That's about a quarter of the between-chart spread of 1.87, which is exactly the gap the two-spread design is for.

4.4 Calibration

Of the example user's 108 stored plays with an earlier best on record, 92 match a chart in the database and survive the $z < -4$ cut. Section 3.11 turns them into these corrections:

Measure	Value
Robust spread of z , after shrinkage (scale s)	0.73
Shift of the curve	+0.21
Tier shift, Expert	-0.03
Tier shift, Master	+0.03
Plays within one spread (coverage)	68%
New bests predicted	39%
New bests that happened	50%

This player's runs clustered tighter around the prediction than the model assumed, so both spreads shrink by 27%: σ_c from 1.87 to 1.37 and σ_{run} from 0.46 to 0.34. The runs also landed a little higher than the curve, so every expectation goes up by 0.21. The model predicted a new best on 39% of these plays and 50% were, so it was slightly too cautious for this player.

4.5 Pricing one chart: GIGANTØMAKHIA

The most recent play in the example user's export is GIGANTØMAKHIA (DX Expert, constant 13.0), played on 30 September at 98.8179%, a new best over 98.7526%. The game counts 6 plays of it, and 3 of them are in the recent-play list: 97.8927, 98.7526 and 98.8179, oldest first. Its notation earns seven labels: multi-touch,

trills, trills on the spot, a trill over a slide, spinning round the ring, fan slides and overlapping loop slides. The model prices a replay in six steps.

1. **Rating.** Achievement 98.8179 is S+ ($k = 20.3$): $13.0 \times 0.988179 \times 20.3 = 260.8$, so 260. This is an old-pool chart and the old cutoff is 263, so it doesn't count toward the rating yet.
2. **Curve.** $m = 96.42$ (curve) + 1.02 (Expert offset) + 0.21 (shift) – 0.03 (Expert tier shift) = 97.62.
3. **Spread.** $\sigma = 0.339 \times \sqrt{(1 + 1/6)} = 0.366$, the calibrated run spread widened for a chart played six times.
4. **Centre.** The best, 98.8179, is above m , so the floor applies: $\mu = 98.8179 - 0.5 \times 0.366 = 98.635$. The recent-play list shows the chart rising by 0.925 from its oldest run (97.8927) to its latest, which adds $\min(0.4, 0.25 \times 0.925) = 0.231$. So $\mu = 98.866$, or 98.87 to two places.
5. **Target.** The ceiling is $\min(100.5, 98.87 + 1.0 \times \max(0.366, 1.37)) = 100.23$, which rules out SSS+. SSS has odds $\Phi((98.87 - 100.0) / 0.366) = 0.1\%$ and SS+ has $\Phi((98.87 - 99.5) / 0.366) = 4.2\%$, both under the balanced floor of 16%. SS at 99.0 has $\Phi(-0.37) = 36\%$ and passes. Its rating is $13.0 \times 0.99 \times 20.8 = 267.7$, so 267.
6. **Gain and score.** The chart is outside the old pool, so $\Delta R = 267 - 263 = 4$. With affinity 1.82 and freshness 0.45 (three recent runs), its pick score is $4 \times 0.357 \times 1.82 \times 0.45 = 1.17$.

So the pick reads: one run has a 36% chance of lifting the rating by 4. Without calibration, the spread would be 0.50 and the centre 98.80, which puts the same odds at 34%. Calibration narrows the spread and raises the centre, and here that adds 2 points of probability.

This table shows every rating move on the example user's list at the balanced level, all of them Expert charts, in the order the bot shows them: by ΔR , then by odds. 44 near misses follow them.

Chart	Constant	Best (%)	Target	μ (%)	Plays	ΔR	Odds
ChurrosParlor	12.5	100.3463	SSS+	100.17	8	11	18%
雲外蒼電-DreamingVoltage-	12.2	100.3777	SSS+	100.33	6	10	32%
エータ・ベータ・イータ	12.4	100.4695	SSS+	100.30	23	10	28%
TidalWave	12.3	100.2075	SSS+	100.19	17	10	19%
BigDaddy	12.0	100.2993	SSS+	100.30	25	7	28%
LatentKingdom	12.8	99.8955	SSS	99.72	25	7	21%
CustomizedJustice	13.0	98.8501	SS	98.67	10	7	18%
Cryptarithm	13.0	99.9080	SSS	99.91	23	6	40%
GIGANTØMAKHIA	13.0	98.8179	SS	98.87	6	4	36%
AzureVixen	13.2	99.4861	SS+	99.31	13	4	29%
鬼女紅妖	11.7	100.3134	SSS+	100.31	2	3	33%
DANGEROOOOUSJUNGLE	11.8	100.4519	SSS+	100.45	2	2	45%
larva	12.9	98.8626	SS	98.69	12	2	19%

The 13 moves add 83 points if each is counted against today's pools. Landed one after another, they add 76, because each landing raises a cutoff that the next gain is measured against. The list has no unplayed charts: nothing in the search window had a first-pass estimate of 94 or more that would enter a pool.

CustomizedJustice shows why pool position matters. It sits in the new pool at the cutoff of 260, so raising it to 267 lifts the pool by the full 7. GIGANTØMAKHIA makes the same 260 to 267 jump from outside the old pool, so it only gains the 4 above that pool's cutoff.

4.6 Where the points went

The example user's GIGANTØMAKHIA play has a judgement page. Its counts are below, with the losses from Section 3.12. The chart holds 861 weight units ($441 + 2 \times 63 + 3 \times 62 + 43 + 5 \times 13$), so one unit is $b = 100 / 861 = 0.1161$.

Note type	Notes	Great	Good	Miss	Loss (points)
Tap	441	43	7	3	1.754
Hold	63	2	0	0	0.093
Slide	62	0	0	0	0
Touch	43	0	0	0	0
Break (base)	13	1	0	0	0.116
Break bonus, 7 perfects and 1 great					0.219
Total					2.182

Tap notes lose $(43/5 + 7/2 + 3) \times 0.1161 = 1.754$. The counts settle everything except the grade of the one break great and how many of the 7 break perfects kept three quarters of their bonus slice instead of half. Those have to cover the 0.289 points the rest leaves unexplained, and only one mix does. The great kept 80% of its break, which costs one unit (0.116). Five of the 7 perfects kept three quarters, so the perfects gave up $(7 \times \frac{1}{2} - 5 \times \frac{1}{4}) / 13 = 0.173$ of the bonus and the great gave up its $0.6 / 13 = 0.046$. That mix lands 0.00004 short of $101 - 98.8179$, inside the 0.0001 the shown achievement cuts off, and that sliver is kept with the bonus. So the total of 2.182 comes from the counts and one solved mix of grades, with nothing assigned by default.

On this one play, taps carry 80% of the loss on 51% of the points at stake ($441 / 861$), a tilt of 1.57. Timing shows 35 fast and 18 late, so 34% of timing errors were late. One play isn't a profile, though, and this user's stored judgement pages say something different. Of 107, three have counts that can't give their recorded achievement and are left out (Section 3.12), which leaves 104:

Note type	Notes	Share of points at stake	Share of loss	Tilt	Critical or perfect
Tap	45,159	44.0%	43.6%	0.99	86.5%
Hold	6,156	11.8%	10.7%	0.90	89.1%
Slide	5,414	15.7%	12.3%	0.78	90.8%
Touch	4,203	4.1%	4.6%	1.14	89.9%
Break	5,616	24.3%	26.6%	1.09	88.8%

The loss shares don't reach 100% because the break bonus, 2.2% of all loss, is reported apart. No type comes near the 1.5 tilt that would name a weakness. The highest are touch (1.14) and break (1.09), so none gets named. Across the 104 plays the user lost 8.93 points a play, 0.20 of it the critical-only break bonus. Late hits make up 51.2% of timing errors (1,837 late against 1,748 fast). As traits (Section 3.14), the note types sit between -0.25 (break) and $+0.28$ (slide) points a play. Both ends reach the 0.25 of a leaning, and neither gets to the 0.4 needed to be confirmed.

4.7 Traits, including simai measures

The notation of 437 of the example user's 554 charts was read, 240 of the 299 at level 12 and above. With those labels, the trait fit measures 57 groups, the 5 note types included. Eight of them are leanings, the ones the Traits view shows as unconfirmed:

Group	Kind	Offset (points)	Charts	Plays	p
reaching across the screen	simai	-0.89	62	19	0.017
slides fired from one spot	simai	-0.84	45	19	0.030
trills	simai	-0.75	79	45	0.080
long streams	simai	-0.70	44	17	0.080
break notes	note type	-0.25	-	104	-
slide notes	note type	+0.28	-	104	-
bursts	simai	+0.90	34	29	0.067
a slide traced straight back	simai	+0.94	63	30	0.022

None is confirmed. Reaching across the screen passes the chance test ($p = 0.017$) but not the split-half stability test, the other simai groups miss the chance test, and the two note types fall short of the 0.4 a note type needs. All six chart-pattern leanings come from the notation, so without it this user wouldn't have any chart-pattern leanings at all. Four groups do pass both tests: charts by 緑風 犬三郎 (+1.52, 17 charts), BUDDiES and newer songs (+1.03, 287 charts), maimai-era songs before DX (-1.30, 61 charts) and charts by 鳩ホルダー (-1.39, 24 charts). Designer and era aren't skills, so they're measured but never reported as traits.

4.8 Progress, forecast and route

The example user's stored readings run from 13,478 on 3 September to 13,580 on 30 September, a gain of 102 points over 27 days and 38 plays (555 to 593). The rating sat at 13,551 from 7 to 22 September, then rose to 13,559 on the 24th and to 13,570 and 13,580 on the 30th. The forecast from Section 3.15 returns 1.79 points per day, 12.5 per week or 54 per month. At that rate the next milestone, 14,000 (gold), lands around 22 May 2027, 235 days after the recording. The window is short, 26 readings over 26 days, so treat that as a rough guide.

The route planner at the balanced level agrees that 14,000 is a long way off. Its route takes 34 steps, 28 of them on charts the user has never played, including the 8 added at the end. It adds 226 rating at the listed targets and still falls 194 short. Weighted by each step's odds, the route is worth about 53 points. The next thousand has to wait for the skill curve itself to rise. No set of targets available today gets there.

5. Discussion

5.1 Design choices

Cautious centres. I'd rather the model under-promise than over-promise. The centre of a prediction never sits above a proven score, except through a recent upward trend capped at 0.4. The curve is capped by what the player has scored nearby, and S-or-better targets need a score like it on a chart at most 0.7 constants easier. Each of those rules is there because a more optimistic version made targets look cheap and then missed on held-out plays.

Two spreads. Keeping the between-chart spread (1.37 after calibration in the example) apart from the run-to-run spread (0.34) is the most important statistical decision in the model. One number can't do both jobs. The first is how much charts differ from each other, and the second is how much one run differs from the next. Pricing a replay with the first made every rank look reachable.

Medians and shrinkage. Medians, quartiles and median absolute deviations shrug off abandoned runs. Every estimate that rests on a small sample, including tier offsets, run spread, calibration and traits, is pulled toward a neutral value by a factor of the form $n / (n + k)$. A new player sees a cautious model, and someone who's been playing for years sees one that follows their data.

Reading the notes. Editors' tags cover about one chart in ten. Measures read from simai notation reach every chart that has a file, and each named measure has to rise with level and agree with the editors wherever they've tagged. In the example, every skill leaning came from the notation.

Tests against chance. A trait only gets named if it beats shuffled tags and keeps its sign in both halves of the player's own charts. Most players see few confirmed traits at first, and the example user has none. That's on purpose. A trait shown for every player wouldn't tell anyone anything.

Recorded plays as a check. The bot keeps every play it sees, so each future play tests an earlier prediction. Calibration, coverage and the predicted-against-actual new-best rate turn that into a running score for the model. For the example user it found runs tighter than assumed and new bests more common than predicted, 50% against 39%.

Other players stay out of the model. The model reads only the player's own scores. The two cohort estimates in Section 3.16, and the medians and judgement comparison shown beside them, are separate on purpose: off by default, display only, nothing in the picks or the route changes because of them. They need at least 20 players before they say anything and they never show who a number came from. A player can leave the count under Sharing.

5.2 Limitations

- **External data.** Constants, versions, regions, notation and tags come from community sources, so a wrong constant changes a chart's rating and every number built on it. For the example user that didn't happen: all 554 scored charts had a published constant, none needed the middle-of-band estimate, and otoge-db and dxrating give the same constant for every one of them. Across the seven other accounts in the debug exports, none of 6,175 scored charts lacked a constant. dxrating disagreed with the stored constant on 4 charts of one account, and none of the 4 would change its best-50. The estimate stays as a fallback for charts the sources haven't rated yet.
- **Normal model.** Run scores are treated as normally distributed around μ and capped at 100.5, so the tail near the cap is only approximate. Near the centre the shape holds up: after calibration, 68% of the example user's recorded plays landed within one spread of the prediction, against 68.3% for a normal distribution. Four other players in the debug exports had enough recorded plays to calibrate (16 to 46 each), and their coverage ran from 62% to 79%. New bests came more often than predicted for the example user (50% against 39%) and for three of those four (for instance 65% against 48%), so the model tends to be cautious at the top end.
- **One run, not many.** The odds are for a single run. If runs were independent, the chance of reaching the target at least once in k runs would be $1 - (1 - P)^k$. For GIGANTØMAKHIA, at $P = 0.357$ per run, that's 73% over three runs and 89% over five. Runs in one session move together with warm-up and fatigue, so the real multi-run chance is probably somewhat lower. The bot only shows the single-run odds.
- **Thin data.** Charts above the player's dense ceiling rest on few scores, and the model widens its spread there instead of claiming precision it doesn't have. For the example user the dense ceiling is 15.0, the top of the constant scale, so none of their charts falls in that region. That's not true of everyone: one player in the debug exports has a dense ceiling of 13.5 but has played up to 14.4, so their hardest charts are priced with the wider spread.
- **Judgement grades.** The page doesn't show which grade a break great or a plain break perfect was, so losses.py works the mix out from the achievement (Section 3.12). Over 457 recorded plays in the example and debug exports, one mix fits on 454, and the greats' cost is unique on 255 of the 263 plays with a break great. On the other 8 it can move by up to 0.10 points between the greats and the bonus. The three plays that fit no mix are all in the example user's data, for instance a 26.33% run whose page shows only five misses. Their counts can't give their achievement, so the play record itself is inconsistent, and the judgement profile leaves those plays out. An earlier version of losses.py charged break greats nothing from the base and assumed every plain perfect lost half its slice. Over the same

plays, its losses came to more than the play lost on 185 of 457, and it reported what break greats cost as bonus. The version described here replaced it during this study.

- **The reproduction here.** Section 4 runs the full profile build, but it reads notation from Maichart-Converts instead of maiノーツ, so it runs without maiノーツ's editor tags, its check on notation note counts, and the optional beta taste model and cohort estimates. The dashboard's numbers can differ where those matter.
- **The cohort estimates.** I haven't run them on a real cohort, only on a simulated world with a planted answer, so how well they do on real players is untested. Observed difficulty measures how hard a chart is for the people who played it, and people play charts that suit them, so a chart that suits its players can look easier than it is. Its standard error leaves out the model's own limits, so in a big cohort it's too small. Players like you rests on neighbours who picked their own charts: in the simulation its median overshoot what the player then scored on the offered charts by 0.5 to 1.7 points. Both are weakest on charts few players have, which is where the minimums matter. The judgement comparison only covers plays whose judgement page was read, which means plays that went through the recent list, and its loss per 100 notes leans on chart length.
- **A one-point gap.** The recomputed best-50 (13,579) is one under the site's 13,580. Two of the seven other accounts in the debug exports also come out exactly one point under the game, and the bot's own stored totals show the same. The other five match exactly. I ruled out several causes:
 - every stored rating agrees with an exact recomputation;
 - the constant sources agree on every chart that counts;
 - no scored chart was left out of the export;
 - moving one chart between the new and old pools explains at most one of the three gaps;
 - no single change to a rank's coefficient closes them.

The shortfall is always one point and never more, so it's small next to any pick's gain, but I still don't know where it comes from.

Rerunning the backtests. Four figures in this paper come from the project's own backtests, as recorded in its source comments. I reran three of them on the example user's data for this section:

- **Headroom** (Section 3.8). Over 246 runs, the source found $h = 0$ better than $h = 0.3$: 64% against 61% of exact ranks called right on runs that beat the best, and 42% against 39% on runs that didn't. On the example user's recorded plays the two settings tie: 44% for both (20 of 45) on runs that beat the best, and 50% (22 of 44) against 49% (21 of 43) on runs that didn't. One player's plays are too few to tell them apart, so the choice of $h = 0$ rests on the source's bigger sample and its reasoning, not on this rerun.
- **Level correlations** (Section 3.13). Over the 6,995 charts read from Maichart-Converts, every named measure still rises with level, but most rise a little less than the source reports: streams +0.53 (source +0.61), trills +0.49 (+0.55), circles +0.46 (+0.60), WiFi slides +0.35 (+0.48) and touch sweeps +0.30 (+0.43). Jacks is still the weakest, at +0.07 (+0.03). The source measured a different notation set, which may explain part of the difference.
- **Title matching** (Section 3.13). Against the same notation set, 56% of the example user's scored charts matched exactly and 84% with the loose match. The source reports 62% and 95% against maiノーツ. The loose match adds about as much here (28 points against 33), but the smaller catalogue leaves more titles unmatched.

The fourth, the first-pass scatter (1.8 times wider), I didn't rerun, so it rests on the source's own measurement.

6. Conclusion

Rasmai rests on one exact fact, the game's rating formula, and estimates everything else from the player's own history. The formula fixes what a score is worth, and the two-pool total fixes which scores count. A skill curve fitted to the player's scores says what they reach at each difficulty. Two separate spreads say how sure the model is about a chart it has never seen and about one it knows, and calibration against recorded plays keeps both in check. Odds times best-50 gain then ranks every chart by what a replay is worth. Notation read from simai stretches the trait analysis from the tenth of charts editors have tagged to most of the game. On the example user's data, the method reproduces all 554 chart ratings, rebuilds the best-50 to within a point, and prices a replay of GIGANTØMAKHIA at 36% for +4 rating. Its note-level accounting covers all 2.18 points lost on that play, including the grade of its one break great. Two optional beta estimates, observed difficulty and players like you, look at other players in aggregate; I haven't tested them on a real cohort yet.

References

1. Cardin Nguyen (Ne-k), [Rasmai](#), source code (GPL-3.0); site at [rasmai.lol](#).
2. SEGA, [maimai DX](#) and [maimai DX NET \(international\)](#).
3. zvuc, [otoge-db](#): chart constants, versions and regions.
4. gekichumai, [dxrating](#): the `dxdata.json` file of revised constants, per-chart regions and aliases.
5. arussin, [maimai-chart-browser](#): search aliases and two slide-pattern rules.
6. Neskol, [Maichart-Converts](#): simai notation used in Section 4.
7. [maiノーツ](#): editors' pattern tags and chart notation.
8. SilentBlue RemyWiki, "maimai DX: Rating": rating colour bands.
9. SpiritsUnite, [maimai-score-details](#) (Apache-2.0): note-loss arithmetic.
10. Wikipedia, background on [weighted least squares](#), [kernel regression](#), [median absolute deviation](#), [ridge regression](#), [permutation tests](#) and the [Theil–Sen estimator](#).

The licences and exact file paths of the third-party sources are listed in the repository's [THIRD_PARTY_NOTICES.md](#).

Appendix A. Source map

Topic	Section	File
Rank table, coefficients, rating formula, challenge levels, colour bands	3.1, 3.2, 3.5, 3.9	engine/analysis/rating.py
Chart index, constants, level bands	3.3	engine/analysis/charts.py
Enrichment, best-50 pools, marginal gain	3.3, 3.4	engine/analysis/pools.py
Skill-curve fit	3.6	engine/analysis/profile/fit.py
Curve, spreads, chart expectation, dropped bests	3.6 to 3.8	engine/analysis/profile/model.py
Profile assembly, tier offsets, current version	3.3, 3.6	engine/analysis/profile/build.py
Target ranks and freshness	3.9, 3.10	engine/analysis/picks/targets.py
Pick ordering and near misses	3.10	engine/analysis/picks/generate.py
Unplayed charts	3.10	engine/analysis/picks/candidates.py
Run spread and calibration	3.7, 3.11	engine/insights/calibration.py
Note-level losses	3.12	engine/losses.py
Judgement profile and note-type traits	3.12, 3.14	engine/judgements.py
simai parsing	3.13	engine/simai/parse.py
simai measures, labels and thresholds	3.13	engine/simai/features.py
simai technique detectors	3.13	engine/simai/techniques.py
Notation reading, caching and matching	3.13	scraping/simai.py , scraping/simai_bulk.py
Chart attributes used as trait tags	3.14	engine/insights/tags.py
Chart traits	3.14	engine/insights/traits.py
Observed difficulty and Players like you	3.16	engine/cohort.py
The hourly cohort and its minimums	3.16	bot/state/cohort.py
Rating forecast	3.15	engine/insights/forecast.py
First-pass prediction and rank ladder	3.8	engine/analysis/planning/predict.py
Route planner	3.15	engine/analysis/planning/build.py
Replaying an export through the model	4	tools/replay.py